

openagent-infra — Datasheet

Reference document for building on top of openagent-infra.

Intended audience: **openagent-api** (the only consumer in the OpenAgent system) and any other service that needs to understand what openagent-infra is, what it owns, and how it is called.

Quick Reference

Item	Value
Role	Model inference proxy for the OpenAgent system
Base URL	<code>http://localhost:8002</code>
Protocol	HTTP/1.1
Streaming	Server-Sent Events (SSE)
Auth in (caller)	<code>X-API-Key</code> header (required on <code>/chat</code>)
Auth out (provider)	<code>Authorization: Bearer PROVIDER_API_KEY</code>
Content type in	<code>application/json</code>
Content type out	<code>text/event-stream</code>
Request format	OpenAI messages format
Reasoning effort	<code>low</code> / <code>medium</code> / <code>high</code> (optional field, default: <code>medium</code>)
Model selection	<code>base</code> (default) / <code>nervous_system</code> (optional field)
Chat endpoint	<code>POST /chat</code>
Health endpoint	<code>GET /health</code>
Docs UI	<code>GET /docs</code>
Caller	openagent-api (the only consumer)
Backend	BYOC provider — any OpenAI-compatible endpoint(s)
Version	1.0.0

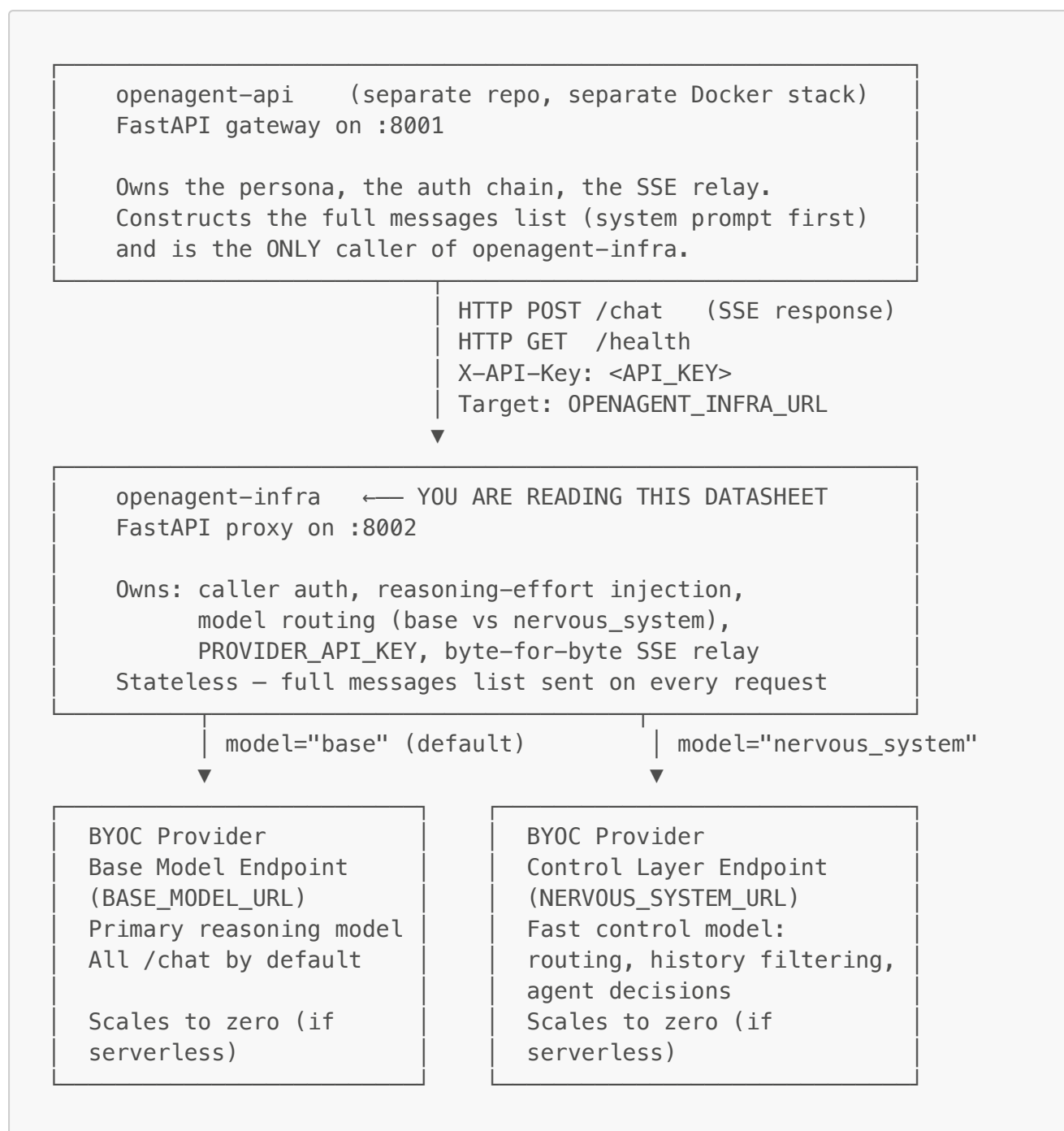
Overview

openagent-infra is the **model inference proxy** of the OpenAgent system. It sits between **openagent-api** (its only caller) and one or two external compute providers, and does three things: authenticates the caller, injects the per-request reasoning effort level into the system message, and streams the provider's response back as Server-Sent Events.

It is deliberately model- and provider-agnostic, built around a **Bring Your Own Compute (BYOC)** approach — it proxies requests to any OpenAI-compatible API endpoint, whether that is a vLLM worker on a serverless host, a local runtime, or a commercial API. The rest of the stack never has to know where inference actually happens.

It is intentionally scoped to the model layer only. It has no knowledge of the OpenAgent persona, the frontend, the conversation state, or the capture layer. The boundary is clean by design: **openagent-infra proxies the models; everything else builds on top of it.**

Where This Service Fits



Port topology:

```
openagent-api (:8001) → openagent-infra (:8002) → BYOC Provider Base
Model [default] → BYOC Provider Control
Layer Model [model="nervous_system"]
```

`openagent-infra` is accessed exclusively by `openagent-api`. The frontend never talks to it, and it never sees `OPENAGENT_API_KEY` (the frontend↔api secret) or any conversation-capture data.

Authentication

Every `POST /chat` request must include a valid API key in the `X-API-Key` header.

```
X-API-Key: your_api_key_here
```

Requests with a missing or invalid key receive `401 Unauthorized`. The `/health` endpoint does not require authentication.

The API key is a shared secret between `openagent-infra` and `openagent-api`, set via `API_KEY` in `openagent-infra`'s `.env`. **It is the same value `openagent-api` holds as `INFRA_API_KEY`** — the naming differs across the boundary (the api side calls it `INFRA_API_KEY`, the infra side calls it `API_KEY`) but the value must match byte-for-byte.

`openagent-infra` uses a second key internally — `PROVIDER_API_KEY` — to authenticate forwarded requests to the BYOC provider endpoint(s) (base model and, when configured, the control-layer model). This key is never exposed to `openagent-api` or any other caller. Two independent secrets at two independent boundaries: `API_KEY` gates who may call `openagent-infra`; `PROVIDER_API_KEY` authenticates `openagent-infra` to the inference backend. Either can be rotated without touching the other.

API Reference

`POST /chat`

The single inference endpoint. Send a full OpenAI messages list and receive a token-by-token streamed response via SSE. Optionally control the reasoning effort level and select the model per request.

Request

```
POST /chat
Content-Type: application/json
X-API-Key: your_api_key_here
```

```
{
  "messages": [
    {"role": "system", "content": "<persona, prepended by openagent-
api>"},
    {"role": "user", "content": "hello"}
  ],
  "reasoning_effort": "medium"
}
```

With reasoning_effort set to high:

```
{
  "messages": [
    {"role": "system", "content": "<persona>"},
    {"role": "user", "content": "Analyze the tradeoffs between SSE and
WebSockets for a streaming chat application"}
  ],
  "reasoning_effort": "high"
}
```

Multi-turn example:

```
{
  "messages": [
    {"role": "system", "content": "<persona>"},
    {"role": "user", "content": "What is the Fibonacci sequence?"},
    {"role": "assistant", "content": "The Fibonacci sequence is..."},
    {"role": "user", "content": "Can you show me in Python?"}
  ],
  "reasoning_effort": "medium"
}
```

Field	Type	Required	Description
<code>messages</code>	array	Yes	Full OpenAI messages list. Cannot be empty. Must contain at least one <code>user</code> message.
<code>messages[].role</code>	string	Yes	One of <code>system</code> , <code>user</code> , <code>assistant</code> .
<code>messages[].content</code>	string	Yes	The message content.
<code>reasoning_effort</code>	string	No	<code>low</code> , <code>medium</code> , or <code>high</code> . Defaults to the server <code>REASONING_EFFORT</code> env var (medium).
<code>model</code>	string	No	<code>base</code> (default) or <code>nervous_system</code> . Routes to the base reasoning model or the control-layer model. Omitting the field always routes to the base model.

Important: `openagent-api` is responsible for constructing the full messages list, including the persona as the first `system` message. `openagent-infra` injects `Reasoning: <level>` into that system message automatically before forwarding to the provider — `openagent-api` does not need to manage the reasoning instruction. This applies to both the base model and the control-layer model.

Reasoning effort guidance

Level	Latency	Use for
low	Fastest	Lightweight tooling calls, simple lookups, routing decisions
medium	Balanced	Standard interactions, general questions (default)
high	Slowest	Complex analysis, multi-step reasoning, hard problems

Response

```
HTTP/1.1 200 OK
Content-Type: text/event-stream; charset=utf-8
Cache-Control: no-cache
Transfer-Encoding: chunked
X-Accel-Buffering: no
```

Each SSE event payload is a JSON-encoded OpenAI ChatCompletion chunk (the format an OpenAI-compatible provider emits natively) — NOT plain text tokens. Chain-of-thought tokens stream first inside `choices[0].delta.reasoning`, then visible answer tokens inside `choices[0].delta.content`, then a final empty-delta chunk with `finish_reason: "stop"`, then the `[DONE]` sentinel.

```
data: {"id":"chatcmpl-...","object":"chat.completion.chunk","choices":
[{"index":0,"delta":
{"role":"assistant","content":"","finish_reason":null}]}

data: {"id":"chatcmpl-...","object":"chat.completion.chunk","choices":
[{"index":0,"delta":{"reasoning":"User"},"finish_reason":null}]}

... (more reasoning tokens – chain-of-thought)

data: {"id":"chatcmpl-...","object":"chat.completion.chunk","choices":
[{"index":0,"delta":{"content":"Hello"},"finish_reason":null}]}

... (more content tokens – visible answer)

data: {"id":"chatcmpl-...","object":"chat.completion.chunk","choices":
[{"index":0,"delta":{"},"finish_reason":"stop"}]}

data: [DONE]
```

`openagent-infra` forwards the provider's SSE stream **byte-for-byte** — it does not decode, re-encode, or interpret the chunk JSON on the relay path. Whatever the OpenAI-compatible provider emits is what the caller receives, terminating with the `[DONE]` sentinel.

Important: The stream always ends with `data: [DONE]`. The caller must watch for this event to know generation is complete.

Important: Both the base model and the control-layer model are reasoning models — each emits a reasoning chain in `delta.reasoning` before the final answer in `delta.content`. Display or filter the reasoning per the caller's UX choice; see the SSE Stream Specification section below.

Error responses

Status	Condition	Body
400	Messages list is empty or contains no user message	<code>{"detail": "Messages list cannot be empty"}</code>
401	X-API-Key header missing or invalid	<code>{"detail": "Invalid or missing API key"}</code>
422	Request body malformed or missing	FastAPI validation error JSON
503	Provider endpoint not reachable	proxy error message

Example — curl (testing only)

```
curl -X POST http://localhost:8002/chat \
  -H "Content-Type: application/json" \
  -H "X-API-Key: your_api_key_here" \
  -d '{"messages": [{"role": "system", "content": "..."}, {"role": "user", "content": "hello"}], "reasoning_effort": "medium"}' \
  --no-buffer
```

GET /health

Lightweight health check. No authentication required. Checks the proxy and both provider endpoints independently.

Request

```
GET /health
```

Response — fully ready

```
{"status": "ok", "proxy": "ok", "base_model": "ok", "nervous_system": "ok"}
```

Response — base_model unreachable / cold-starting

```
{"status": "degraded", "proxy": "ok", "base_model": "unreachable", "nervous_system": "ok"}
```

Response — nervous system not yet configured

```
{"status": "ok", "proxy": "ok", "base_model": "ok", "nervous_system": "not configured"}
```

Note: Always returns HTTP 200. `status` is `ok` when the base model is reachable — the nervous-system model is checked independently and does not affect the top-level status. `not configured` means `NERVOUS_SYSTEM_URL` is not set in `.env`.

`openagent-api` polls this endpoint and translates `degraded` (base model cold-starting) into `loading` for the frontend's gate-open loop. The top-level `status` field is the only field `openagent-api` needs to read; the per-endpoint fields are diagnostic detail.

```
curl http://localhost:8002/health
```

GET /docs

Auto-generated Swagger UI. For interactive testing only.

```
http://localhost:8002/docs
```

System Prompt

The system prompt — the persona — is owned by `openagent-api`, not `openagent-infra`. `openagent-api` loads it once at startup and sends it as the first `system` message in the messages list on every request.

`openagent-infra` automatically appends `Reasoning: <level>` to that system message before forwarding to the provider — `openagent-api` does not include the reasoning instruction manually.

`openagent-infra` has no knowledge of what the system prompt contains. It receives the full messages list, injects the reasoning level, and forwards. The persona text passes through untouched.

openagent-api is responsible for:

- Owning and managing the persona (`src/prompt/bio.txt`, baked into its image).
- Including it as `{"role": "system", "content": "..."} as the first message on every /chat request.`
- Setting `reasoning_effort` per request based on the use case (or omitting it to let `openagent-infra` default).

SSE Stream Specification

Event format

```
data: <JSON ChatCompletion chunk>\n\n
```

Each event payload is a JSON-encoded OpenAI ChatCompletion chunk. The double newline `\n\n` terminates each event.

Stream lifecycle



Handling the reasoning chain

Both the base model and the control-layer model are reasoning models. Every response from either model includes a reasoning chain (in `delta.reasoning`) before the final answer (in `delta.content`).

Three options for handling on the caller side:

Option 1 — Hide entirely: Filter `delta.reasoning` tokens before displaying. Show only `delta.content`.

Option 2 — Show in a collapsible: Display reasoning in a collapsed "Show thinking" section, answer in the main surface. (This is what `openagent-frontend` does.)

Option 3 — Show everything: Stream all tokens directly to the UI as-is.

The choice is entirely a caller/frontend decision. `openagent-infra` streams everything byte-for-byte and takes no position on display.

Technical Specifications

Models

`openagent-infra` routes between two logical models. Both are reasoning models that emit a reasoning chain before their answer, and both are reached over an OpenAI-compatible API — the actual weights, quantization, parameter counts, context window, and serving runtime are entirely the BYOC provider's concern and are not pinned by `openagent-infra`.

base model (primary)

Property	Value
Selector	<code>model="base"</code> (default; also the value when <code>model</code> is omitted)
Endpoint	<code>BASE_MODEL_URL</code>
Type	Reasoning model — emits a reasoning chain before the answer
API	OpenAI-compatible (<code>/chat/completions</code> , SSE streaming)
Role	Default model — all everyday OpenAgent conversations

nervous-system model (control layer)

Property	Value
Selector	<code>model="nervous_system"</code>
Endpoint	<code>NERVOUS_SYSTEM_URL</code> (optional — when unset, the route is "not configured")
Type	Reasoning model — emits a reasoning chain before the answer
API	OpenAI-compatible (<code>/chat/completions</code> , SSE streaming)

Property	Value
Role	Fast control layer — routing, history filtering, agent decisions

Because the models are reached purely as OpenAI-compatible endpoints, you can point the two URLs at the same provider or different providers, at the same model in two sizes or two entirely different models, balancing compute cost against latency as you see fit. `openagent-infra` does not care.

Generation parameters

Parameter	Value	Notes
<code>reasoning_effort</code>	<code>low</code> / <code>medium</code> / <code>high</code>	First-class API field — controls reasoning depth and latency. Injected into the system message as <code>Reasoning: <level></code> .
Batching	Provider-dependent	Continuous batching, concurrency, and queueing are the provider's concern.

Infrastructure

Property	Value
Inference provider	BYOC — any OpenAI-compatible endpoint(s) (e.g. RunPod, OpenAI, a local vLLM/Ollama runtime)
Base model endpoint	<code>BASE_MODEL_URL</code>
Control-layer endpoint	<code>NERVOUS_SYSTEM_URL</code> (optional)
Endpoint format	OpenAI-compatible chat-completions API
Scaling	Provider-dependent; serverless endpoints scale to zero independently when idle

`openagent-infra` itself runs no model and holds no weights — it is a thin async proxy. All GPU/compute concerns live with the provider.

Container

Property	Value
Base image	<code>python:3.12-slim</code>
WORKDIR	<code>/app</code>
Proxy port	<code>8002</code> (public)
Env file	<code>.env</code> at project root

Property	Value
GPU required	No — inference runs at the BYOC provider
Volume required	No — no weights, no state

Startup timing

Phase	Approximate duration
Proxy startup	< 10 seconds (no model to load)
Provider cold start (serverless)	Provider-dependent; can be minutes for a worker spinning up from zero
Provider warm	Provider-dependent; typically seconds

If your BYOC endpoints are serverless and scale to zero, the first request after an idle period waits for the provider's worker to spin up. The two endpoints have independent cold-start cycles. Design the caller with a loading state for this case — `openagent-infra` reports `degraded` on `/health` while the base model is cold-starting and `ok` once it is reachable.

Generation timing

Scenario	Reasoning	Approximate duration
Simple greeting	low	5–15 seconds
Short factual question	medium	15–45 seconds
Complex reasoning task	high	1–3 minutes

Actual numbers depend entirely on the provider, the model, and load. Treat these as order-of-magnitude expectations, not guarantees.

Integration Notes for openagent-api

`openagent-api` is the only consumer of `openagent-infra`. The integration points:

Readiness check pattern

Poll `/health` until `status` shows `"ok"`. The response includes `base_model` and `nervous_system` fields independently; `status` is `ok` when the base model is reachable. During a provider cold start this may take a few minutes — `openagent-api` surfaces this to the frontend as `loading`.

```
GET /health → {"status": "degraded", ...} # provider worker cold-starting
GET /health → {"status": "ok", ...}      # ready
```

Selecting the model per request

Pass `model` in the request body to route to a specific endpoint. If omitted, every request routes to the base model — the pipeline is unbroken for callers that never set it. `openagent-api` omits the field on every call today (it always uses the base model); the `nervous_system` route is available for callers that need a fast control model.

```
{ "messages": [...], "reasoning_effort": "medium", "model": "base" }  
# default  
{ "messages": [...], "reasoning_effort": "low",    "model":  
  "nervous_system" } # control layer
```

Setting reasoning effort per request

Pass `reasoning_effort` in the request body. If omitted, the server default (`medium`, from the `REASONING_EFFORT` env var) applies. Applies to both models.

Constructing the messages list

`openagent-api` builds the full messages list, including the persona as the first `system` message. `openagent-infra` injects the reasoning level — the caller does not add it.

Token handling

Each SSE event is a JSON ChatCompletion chunk. The caller's decoder must `json.loads()` each `data:` payload and route `choices[0].delta.reasoning` and `choices[0].delta.content` to the appropriate surfaces. The stream terminates with an empty-delta `finish_reason: "stop"` chunk followed by `data: [DONE]`.

API key handling

`API_KEY` is the caller↔infra secret (the same value `openagent-api` holds as `INFRA_API_KEY`). `PROVIDER_API_KEY` never leaves `openagent-infra` — `openagent-api` does not need it and never sees it.

Cold start handling

Serverless provider endpoints scale to zero when idle. The first request after inactivity triggers a cold start. `/health` returns `degraded` during the cold start and `ok` once the worker is ready.

Long generation times

Even at `low` effort, generation takes several seconds. The caller should not set a short read timeout — `openagent-api` uses an unbounded read timeout on this boundary for exactly this reason.

CORS

`openagent-infra` does not configure CORS headers. It is called server-side by `openagent-api`, never directly from a browser. This keeps `PROVIDER_API_KEY` server-side and out of any client.

The `[DONE]` sentinel

Always handle `[DONE]` explicitly — it signals generation is complete and the connection can be closed.

401 handling

A `401` indicates a key configuration error between `openagent-api` and `openagent-infra` (`INFRA_API_KEY` on the api side does not match `API_KEY` on the infra side). Log it for the operator — it is not an end-user-facing error.

Environment Variables Reference

Variable	Type	Default	Description
<code>API_KEY</code>	string	—	Secret validated against the <code>X-API-Key</code> header. Must match <code>openagent-api</code> 's <code>INFRA_API_KEY</code> byte-for-byte. Required.
<code>BASE_MODEL_URL</code>	string	—	OpenAI-compatible endpoint for the base model. Default route for all requests. Required.
<code>NERVOUS_SYSTEM_URL</code>	string	—	OpenAI-compatible endpoint for the control-layer model. Used when <code>model="nervous_system"</code> . Optional — when unset, that route reports "not configured".
<code>PROVIDER_API_KEY</code>	string	—	Bearer credential for the BYOC provider endpoint(s). Never exposed to callers. Required.
<code>REASONING_EFFORT</code>	string	<code>medium</code>	Server default reasoning level. Overridable per request.

Known Behaviors

Behavior	Cause	Caller handling
Reasoning tokens before the final answer	Both models are reasoning models — they reason before answering	Filter or display per UX choice (reasoning is in <code>delta.reasoning</code> , answer in <code>delta.content</code>)
<code>low</code> effort is faster but shallower	Less reasoning chain generated	Use for lightweight calls only
<code>high</code> effort significantly slower	Extended reasoning chain	Show a loading indicator; no short read timeout

Behavior	Cause	Caller handling
<code>degraded</code> on <code>/health</code> during cold start	Provider worker scaling up from zero	Poll until <code>status: ok</code> — may take a few minutes on serverless
<code>degraded</code> on <code>/health</code> otherwise	Provider endpoint unreachable	Check the provider's console for endpoint/worker status
<code>401</code> on a valid-looking request	<code>API_KEY</code> (infra) and <code>INFRA_API_KEY</code> (api) mismatch	Configuration error — align the two values
<code>422</code> on a malformed request	Pydantic validation failed	Ensure <code>messages</code> array with valid <code>role</code> and <code>content</code> fields
<code>400</code> missing user message	No <code>user</code> -role message in <code>messages</code>	Always include at least one user message
<code>nervous_system</code> routes to the control model	<code>model="nervous_system"</code> field set	Confirm <code>NERVOUS_SYSTEM_URL</code> is set in <code>.env</code>
<code>base_model</code> shows unreachable in <code>/health</code>	Base endpoint cold-starting or down	Poll <code>/health</code> until <code>base_model</code> is <code>ok</code>
<code>nervous_system</code> shows not configured	<code>NERVOUS_SYSTEM_URL</code> not set in <code>.env</code>	Add the endpoint URL once the control model is deployed
Reasoning-format delimiters occasionally in the stream	Some provider serving runtimes don't fully parse the model's reasoning format	<code>openagent-infra</code> forwards bytes unchanged; the caller's display policy handles it. Resolves when the provider's runtime supports the model's reasoning parser.

Design Decisions

Why keep a FastAPI proxy instead of calling providers directly?

A provider usually offers a single shared API key. Keeping FastAPI as a proxy puts caller authentication in one place and keeps the provider's billing credential out of the gateway and frontend layers. It also gives a stable, OpenAI-shaped contract regardless of which provider sits behind it.

Why two separate API keys?

`API_KEY` authenticates the caller (`openagent-api`) to `openagent-infra`. `PROVIDER_API_KEY` authenticates `openagent-infra` to the inference backend. Separating them means the caller key can be rotated without touching the provider configuration, and the provider key never leaves this service.

Why reasoning effort as an API field?

Both models support configurable reasoning depth. Exposing it per request lets a frontend offer Quick / Standard / Deep modes and lets tooling set depth per use case. The proxy injects it into the system message so callers never hand-write the instruction.

Why the OpenAI messages format?

It makes `openagent-infra` compatible with almost any frontend and model backend, keeps the serving layer stateless, and keeps the wire protocol standard and boring — which is what you want in an inference proxy.

Why byte-for-byte SSE relay?

Anything the proxy parses on the relay path, it can break. Forwarding the provider's stream unchanged keeps the proxy simple and lets the caller's decoder be the single place that understands the chunk format.

Why port 8002?

Port convention: 8000 = `openagent-frontend`, 8001 = `openagent-api`, 8002 = `openagent-infra`, 8003 = `openagent-logger`. The numbering reflects the request flow.

openagent-infra — part of the OpenAgent system